

Assignment – 17.2

Name: Saisharan

Ht.no: 2303A51434

bt.no: 21

Task Description – 1(Student Academic Data Cleaning)

- Task:

Use AI assistance to generate a Python script that cleans and prepares a student academic dataset for analysis.

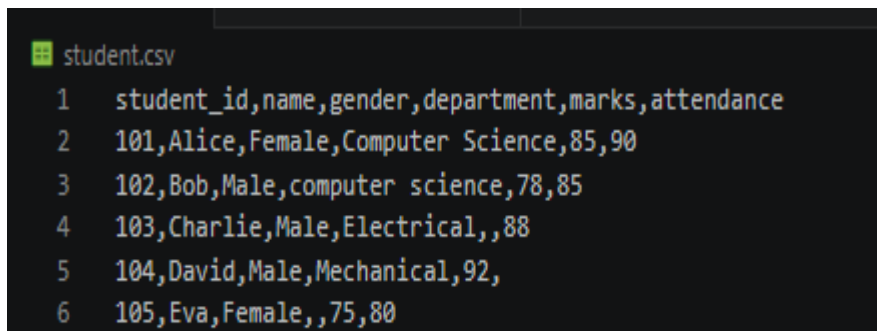
Instructions:

- Handle missing values in marks, attendance, and department.
- Remove duplicate student records based on student ID.
- Standardize text fields such as department names.
- Encode categorical variables like gender and department.

Expected Output:

- A cleaned and encoded Pandas DataFrame suitable for academic performance analysis.

Dataset file :



```
student.csv
1 student_id,name,gender,department,marks,attendance
2 101,Alice,Female,Computer Science,85,90
3 102,Bob,Male,computer science,78,85
4 103,Charlie,Male,Electrical,,88
5 104,David,Male,Mechanical,92,
6 105,Eva,Female,,75,80
```

Code screenshot:

```

EXPLORER
  AI19.2
    cleaned_students.csv
    cleaned_students.py
    student.csv
  cleaned_students.py > ...
1  import pandas as pd
2
3  # Load dataset
4  data = pd.read_csv("student.csv")
5
6  print("Original Data:\n", data)
7
8  # -----
9  # 1. Handle Missing Values
10 # -----
11
12 # Fill missing marks with average marks
13 data['marks'] = data['marks'].fillna(data['marks'].mean())
14
15 # Fill missing attendance with average attendance
16 data['attendance'] = data['attendance'].fillna(data['attendance'].mean())
17
18 # Fill missing department with 'Unknown'
19 data['department'] = data['department'].fillna('Unknown')
20
21 # -----
22 # 2. Remove Duplicate Records
23 # -----
24
25 data = data.drop_duplicates(subset='student_id')
26
27 # -----
28 # 3. Standardize Text Fields
29 # -----
30
31 # Convert department names to lowercase then capitalize properly
32 data['department'] = data['department'].str.strip().str.lower()
33
34 # Standardize common department names
35 data['department'] = data['department'].replace({
36     'computer science': 'Computer Science',
37     'electrical': 'Electrical',
38     'mechanical': 'Mechanical',
39     'civil': 'Civil',
40     'unknown': 'Unknown'
41 })
42
43 # -----
44 # 4. Encode Categorical Variables
45 # -----
46
47 # Encode gender (Male=0, Female=1)
48 data['gender'] = data['gender'].map({'Male': 0, 'Female': 1})
49
50 # One-hot encoding for department
51 data = pd.get_dummies(data, columns=['department'])

```

Output screenshot:

```

PS C:\Users\phani\OneDrive\Desktop\Btech\3-2\Ai19.2> python cleaned_students.py
Original Data:
  student_id  name  gender  department  marks  attendance
0         101  Alice  Female  Computer Science  85.0         90.0
1         102   Bob   Male   computer science  78.0         85.0
2         103  Charlie  Male     Electrical   NaN         88.0
3         104   David  Male     Mechanical  92.0          NaN
4         105    Eva  Female         NaN   75.0         80.0

Cleaned Data:
  student_id  name  gender  ...  department_Electrical  department_Mechanical  department_Unknown
0         101  Alice     1  ...                    False                    False                    False
1         102   Bob     0  ...                    False                    False                    False
2         103  Charlie  0  ...                     True                    False                    False
3         104   David  0  ...                    False                     True                    False
4         105    Eva     1  ...                    False                    False                     True

[5 rows x 9 columns]
Cleaned dataset saved as 'cleaned_students.csv'

```

Task Description – 2 (Financial Transaction Data Preprocessing)

• Task:

Preprocess financial transaction data using AI-generated Python code.

Instructions:

- Convert transaction date columns to datetime format.

- Create derived features such as transaction month and year.
- Normalize the transaction amount column.
- Identify and handle extreme transaction values.

Expected Output:

- A transformed dataset with normalized values and newly generated time-based features.

Dataset file:

```
transactions.csv
1 transaction_id,user_id,transaction_date,amount,transaction_type
2 1,1001,2024-01-15,5000,Debit
3 2,1002,2024/02/20,12000,Credit
4 3,1003,15-03-2024,300,Debit
5 4,1004,2024-04-10,999999,Credit
6 5,1005,,4500,Debit
7 |
```

Code

screenshot:

```
process_transactions.py > ...
import pandas as pd
from sklearn.preprocessing import MinMaxScaler
import numpy as np
# Load dataset
transactions = pd.read_csv("transactions.csv")
print("Original Data:\n", transactions)
# -----
# 1. Convert transaction_date to datetime
# -----

transactions['transaction_date'] = pd.to_datetime(
    transactions['transaction_date'],
    errors='coerce'
)
# Fill missing dates with forward fill
transactions['transaction_date'] = transactions['transaction_date'].fillna(method='ffill')

# -----
# 2. Create Derived Features
# -----

transactions['transaction_year'] = transactions['transaction_date'].dt.year
transactions['transaction_month'] = transactions['transaction_date'].dt.month

# -----
# 3. Handle Extreme Values (Outliers)
# -----

# Remove negative transactions (optional rule)
transactions['amount'] = transactions['amount'].apply(lambda x: abs(x))
# Cap extreme values using IQR method
Q1 = transactions['amount'].quantile(0.25)
Q3 = transactions['amount'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

transactions['amount'] = np.where(
    transactions['amount'] > upper_bound,
    upper_bound,
    transactions['amount']
)

transactions['amount'] = np.where(
    transactions['amount'] < lower_bound,
    lower_bound,
    transactions['amount']
)
```

```

)

# -----
# 4. Normalize Transaction Amount
# -----

scaler = MinMaxScaler()
transactions['normalized_amount'] = scaler.fit_transform(transactions[['amount']])

# -----
# Final Output
# -----

print("\nProcessed Data:\n", transactions)

# Save cleaned dataset
transactions.to_csv("processed_transactions.csv", index=False)

print("\nProcessed dataset saved as 'processed_transactions.csv'")

```

Output screenshot:

```

PS C:\Users\phani\OneDrive\Desktop\Btech\3-2\Ai19.2> python preprocess_transactions.py
Original Data:
  transaction_id  user_id transaction_date  amount transaction_type
0              1     1001      2024-01-15    5000           Debit
1              2     1002      2024/02/20   12000           Credit
2              3     1003      15-03-2024     300           Debit
3              4     1004      2024-04-10  999999           Credit
4              5     1005           NaN    4500           Debit

Processed Data:
  transaction_id  user_id transaction_date  ... transaction_year transaction_month  normalized_amount
0              1     1001      2024-01-15  ...             2024                1             0.204793
1              2     1002      2024-01-15  ...             2024                1             0.509804
2              3     1003      2024-01-15  ...             2024                1             0.000000
3              4     1004      2024-04-10  ...             2024                4             1.000000
4              5     1005      2024-04-10  ...             2024                4             0.183007

[5 rows x 8 columns]

```

Task 3 – (Environmental Sensor Data Preparation)

- Task:

Clean and preprocess environmental sensor data using AI-assisted scripts.

Instructions:

- Handle missing values in temperature, humidity, and air_quality_index.
- Scale numerical features using standard scaling.
- Encode categorical fields such as sensor_status.
- Split the dataset into training and testing subsets.

Dataset file:

```
sensor_data.csv
1  sensor_id,temperature,humidity,air_quality_index,sensor_status
2  1,30.5,65,120,Active
3  2,28.0,70,,Inactive
4  3,,60,90,Active
5  4,35.2,,150,Faulty
6  5,29.8,75,110,Active
```

Code screenshot:'

```
preprocess_sensor_data.py 7...
1  import pandas as pd
2  from sklearn.preprocessing import StandardScaler
3  from sklearn.model_selection import train_test_split
4
5  # Load dataset
6  data = pd.read_csv("sensor_data.csv")
7
8  print("Original Data:\n", data)
9
10 # -----
11 # 1. Handle Missing Values
12 # -----
13
14 # Fill missing numerical values with mean
15 data['temperature'] = data['temperature'].fillna(data['temperature'].mean())
16 data['humidity'] = data['humidity'].fillna(data['humidity'].mean())
17 data['air_quality_index'] = data['air_quality_index'].fillna(data['air_quality_index'].mean())
18
19 # -----
20 # 2. Encode Categorical Variables
21 # -----
22
23 # Convert sensor_status to numeric (One-hot encoding)
24 data = pd.get_dummies(data, columns=['sensor_status'])
25
26 # -----
27 # 3. Scale Numerical Features
28 # -----
29
30 scaler = StandardScaler()
31
32 numerical_cols = ['temperature', 'humidity', 'air_quality_index']
33 data[numerical_cols] = scaler.fit_transform(data[numerical_cols])
34
35 # -----
36 # 4. Split Dataset
37 # -----
38
39 X = data.drop('sensor_id', axis=1)
40 y = data['sensor_id'] # just for example (usually target variable)
41
42 X_train, X_test, y_train, y_test = train_test_split(
43     X, y, test_size=0.2, random_state=42
44 )
45
46 # -----
47 # Final Output
48 # -----
49
50 print("\nProcessed Data:\n", data)
51 print("\nTraining Set:\n", X_train)
52 print("\nTesting Set:\n", X_test)
53
```

Output:

```
Original Data:
  sensor_id  temperature  humidity  air_quality_index  sensor_status
0         1         30.5        65.0             120.0         Active
1         2         28.0        70.0              NaN         Inactive
2         3         NaN        60.0             90.0         Active
3         4         35.2        NaN             150.0         Faulty
4         5         29.8        75.0             110.0         Active

Processed Data:
  sensor_id  temperature  humidity  air_quality_index  sensor_status_Active  sensor_status_Faulty  sensor_status_Inactive
0         1   -0.157715    -0.5         0.129099          True          False          False
1         2   -1.209147    0.5         0.000000          False         False          True
2         3    0.000000   -1.5        -1.420094          True          False         False
3         4    1.818978    0.0         1.678293          False         True         False
4         5   -0.452116    1.5        -0.387298          True          False         False

Training Set:
  temperature  humidity  air_quality_index  sensor_status_Active  sensor_status_Faulty  sensor_status_Inactive
4   -0.452116    1.5        -0.387298          True          False          False
2    0.000000   -1.5        -1.420094          True          False          False
0   -0.157715   -0.5         0.129099          True          False          False
3    1.818978    0.0         1.678293          False         True          False

Testing Set:
  temperature  humidity  air_quality_index  sensor_status_Active  sensor_status_Faulty  sensor_status_Inactive
1   -1.209147    0.5         0.0          False         False          True

Processed dataset saved as 'processed_sensor_data.csv'
```

Task 4 – (Real-Time Application: Smart City Traffic Data Cleaning)

- Scenario:

A smart city system collects traffic data from multiple sensors, resulting in noisy and inconsistent records.

Requirements:

- Standardize road and location names.
- Fill missing traffic density values using appropriate statistical methods.
- Remove duplicate sensor readings.
- Normalize speed and vehicle count metrics.
- Generate a brief summary comparing dataset quality before and after cleaning

Sample Code:

```
import pandas as pd

traffic = pd.read_csv("traffic_data.csv")

print(traffic.describe())
```

Expected Output:

- A cleaned and normalized traffic dataset with an accompanying data-quality improvement summary.

Dataset file:

```
traffic_data.csv
1  sensor_id,road_name,location,traffic_density,speed,vehicle_count
2  1,MG Road,Hyderabad,80,45,200
3  2,mg road,hyderabad,,50,220
4  3,Ring Road,Delhi,120,60,300
5  4,Ring road,delhi,110,,280
6  5,Highway 1,Mumbai,300,80,500
```

Code screenshot:

```

clean_traffic_data.py > ...
1  import pandas as pd
2
3  # Load dataset
4  traffic = pd.read_csv("traffic_data.csv")
5
6  print("Before Cleaning:\n")
7  print(traffic.describe(include='all'))
8
9  # -----
10 # 1. Standardize Text Fields
11 # -----
12
13 traffic['road_name'] = traffic['road_name'].str.strip().str.lower().str.title()
14 traffic['location'] = traffic['location'].str.strip().str.lower().str.title()
15
16 # -----
17 # 2. Handle Missing Values
18 # -----
19
20 # Fill traffic_density with median (better for skewed data)
21 traffic['traffic_density'] = traffic['traffic_density'].fillna(
22     traffic['traffic_density'].median()
23 )
24
25 # Fill speed and vehicle_count with mean
26 traffic['speed'] = traffic['speed'].fillna(traffic['speed'].mean())
27 traffic['vehicle_count'] = traffic['vehicle_count'].fillna(traffic['vehicle_count'].mean())
28
29 # -----
30 # 3. Remove Duplicate Records
31 # -----
32
33 traffic = traffic.drop_duplicates()
34 # -----
35 # 4. Normalize Numerical Features
36 # -----
37
38 for col in ['speed', 'vehicle_count']:
39     min_val = traffic[col].min()
40     max_val = traffic[col].max()
41     traffic[col] = (traffic[col] - min_val) / (max_val - min_val)
42
43 # -----
44 # 5. After Cleaning Summary
45 # -----
46
47 print("\nAfter Cleaning:\n")
48 print(traffic.describe(include='all'))
49
50 # -----
51 # 6. Save Cleaned Data
52 # -----
53 traffic.to_csv("cleaned_traffic_data.csv", index=False)
54 print("\nCleaned dataset saved as 'cleaned_traffic_data.csv'")

```

Output:

```

PS C:\Users\phani\OneDrive\Desktop\Btech\3-2\Ai19.2> & C:/Users/phani/AppData/Local/Microsoft/WindowsApps/python3.11.exe c:/Users/phani/OneDrive/Desktop/Btech/3-2/Ai19.2/clean_traffic_data.p
Before Cleaning:

```

	sensor_id	road_name	location	traffic_density	speed	vehicle_count
count	5.000000	5	5	4.000000	4.000000	5.000000
unique	NaN	5	5	NaN	NaN	NaN
top	NaN	MG Road	Hyderabad	NaN	NaN	NaN
freq	NaN	1	1	NaN	NaN	NaN
mean	3.000000	NaN	NaN	152.500000	58.750000	300.000000
std	1.581139	NaN	NaN	99.791449	15.47848	119.163753
min	1.000000	NaN	NaN	80.000000	45.000000	200.000000
25%	2.000000	NaN	NaN	102.500000	48.750000	220.000000
50%	3.000000	NaN	NaN	115.000000	55.000000	280.000000
75%	4.000000	NaN	NaN	165.000000	65.000000	300.000000
max	5.000000	NaN	NaN	300.000000	80.000000	500.000000

```

After Cleaning:

```

	sensor_id	road_name	location	traffic_density	speed	vehicle_count
count	5.000000	5	5	5.000000	5.000000	5.000000
unique	NaN	3	3	NaN	NaN	NaN
top	NaN	Mg Road	Hyderabad	NaN	NaN	NaN
freq	NaN	2	2	NaN	NaN	NaN
mean	3.000000	NaN	NaN	145.000000	0.392857	0.333333
std	1.581139	NaN	NaN	88.034084	0.382993	0.397213
min	1.000000	NaN	NaN	80.000000	0.000000	0.000000
25%	2.000000	NaN	NaN	110.000000	0.142857	0.066667
50%	3.000000	NaN	NaN	115.000000	0.392857	0.266667
75%	4.000000	NaN	NaN	120.000000	0.428571	0.333333
max	5.000000	NaN	NaN	300.000000	1.000000	1.000000

```

Cleaned dataset saved as 'cleaned_traffic_data.csv'

```

Task 5 – (Social Media Text Data Cleaning and Feature Extraction)

• Task:

Use AI-assisted Python scripting to clean and preprocess social media text data for sentiment analysis.

Instructions:

- Remove duplicate posts and null text entries.
- Clean text by removing URLs, special characters, and extra spaces.
- Convert text to lowercase and perform basic tokenization.
- Remove stopwords and perform stemming or lemmatization.
- Generate basic text features such as word count and sentimentscore.

Sample Code:

```
import pandas as pd

import re

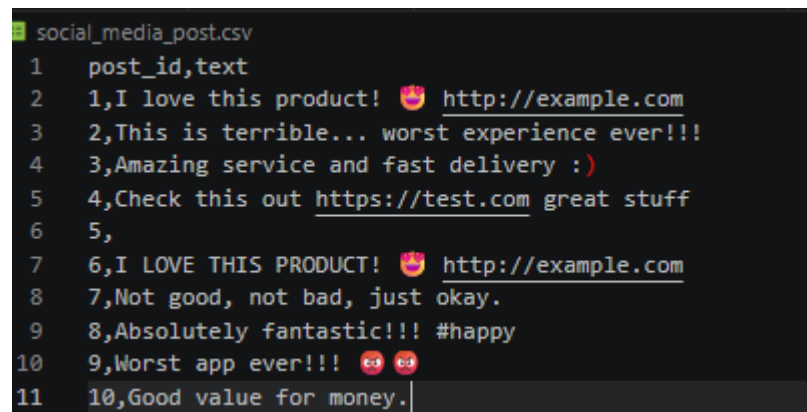
posts = pd.read_csv("social_media_posts.csv")

print(posts.head())
```

Expected Output:

A cleaned text dataset with extracted features suitable for sentiment analysis models.

Dataset file:



```
social_media_post.csv
1 post_id,text
2 1,I love this product! 🥰 http://example.com
3 2,This is terrible... worst experience ever!!!
4 3,Amazing service and fast delivery :)
5 4,Check this out https://test.com great stuff
6 5,
7 6,I LOVE THIS PRODUCT! 🥰 http://example.com
8 7,Not good, not bad, just okay.
9 8,Absolutely fantastic!!! #happy
10 9,Worst app ever!!! 😡 😡
11 10,Good value for money.
```

Code screenshot:

```

preprocess_social_text.py > ...
1  import pandas as pd
2  import re
3  from collections import Counter
4
5  # Load dataset
6  posts = pd.read_csv("social_media_posts.csv")
7
8  print("Original Data:\n", posts)
9
10 # -----
11 # 1. Remove Duplicates & Nulls
12 # -----
13
14 posts = posts.drop_duplicates(subset='text')
15 posts = posts.dropna(subset=['text'])
16
17 # -----
18 # 2. Text Cleaning Function
19 # -----
20
21 def clean_text(text):
22     text = re.sub(r"http\S+", "", text)          # remove URLs
23     text = re.sub(r"[^a-zA-Z\s]", "", text)      # remove special characters
24     text = re.sub(r"\s+", " ", text)            # remove extra spaces
25     return text.strip().lower()                 # lowercase
26
27 posts['clean_text'] = posts['text'].apply(clean_text)
28
29 # -----
30 # 3. Tokenization
31 # -----
32
33 posts['tokens'] = posts['clean_text'].apply(lambda x: x.split())
34
35 # -----
36 # 4. Remove Stopwords
37 # -----
38
39 stopwords = {
40     'is', 'this', 'and', 'the', 'for', 'to', 'a',
41     'of', 'in', 'on', 'not', 'just', 'out'
42 }
43
44 posts['tokens'] = posts['tokens'].apply(
45     lambda words: [w for w in words if w not in stopwords]
46 )
47
48 # -----
49 # 5. Simple Stemming (Manual)
50 # -----
51
52 def simple_stem(word):
53     for suffix in ['ing', 'ly', 'ed', 's']:
54         if word.endswith(suffix):
55             return word[:-len(suffix)]

```

```

preprocess_social_text.py > ...
52 def simple_stem(word):
53     for suffix in ['ing', 'ly', 'ed', 's']:
54         if word.endswith(suffix):
55             return word[:-len(suffix)]
56     return word
57
58 posts['tokens'] = posts['tokens'].apply(
59     lambda words: [simple_stem(w) for w in words]
60 )
61
62 # -----
63 # 6. Feature Extraction
64 # -----
65
66 # Word count
67 posts['word_count'] = posts['tokens'].apply(len)
68
69 # Simple Sentiment Score (basic rule-based)
70 positive_words = {'love', 'amazing', 'fantastic', 'good', 'great', 'happy'}
71 negative_words = {'worst', 'bad', 'terrible'}
72
73 def sentiment_score(words):
74     score = 0
75     for w in words:
76         if w in positive_words:
77             score += 1
78         elif w in negative_words:
79             score -= 1
80     return score
81
82 posts['sentiment_score'] = posts['tokens'].apply(sentiment_score)
83
84 # -----
85 # Final Output
86 # -----
87
88 print("\nProcessed Data:\n", posts[['clean_text', 'tokens', 'word_count', 'sentiment_score']])
89
90 # Save cleaned dataset
91 posts.to_csv("processed_social_media_posts.csv", index=False)
92
93 print("\nProcessed dataset saved as 'processed_social_media_posts.csv'")

```

Output:

```
Original Data:
  post_id      text
0      1  I love this product! 🍷 http://example.com
1      2  This is terrible... worst experience ever!!!
2      3    Amazing service and fast delivery :)
3      4  Check this out https://test.com great stuff
4      5                                     NaN

Processed Data:

      clean_text      tokens  word_count  sentiment_score
0      i love this product  [i, love, product]          3           1
1  this is terrible worst experience ever  [terrible, worst, experience, ever]          4          -2
2    amazing service and fast delivery  [amaz, service, fast, delivery]          4           0
3      check this out great stuff  [check, great, stuff]          3           1
5      i love this product  [i, love, product]          3           1
6    absolutely fantastic happy  [absolute, fantastic, happy]          3           2
7      worst app ever  [worst, app, ever]          3          -1
8    good value for money  [good, value, money]          3           1

✅ File saved as 'processed_social_media_posts.csv'
```